# CERI/CIVL- 8104/8126
# Data Analysis in Geophysics
# -Machine Learning-

In this exercise, you will solve one of the classic introductory machine-learning problems, i.e. classifying objects in images. You will analyze images of hand-written numbers and train an ANN to correctly classify each single digit number. For this problem, we will use a Multi-Layer Perceptron algorithm implemented in Python's scikit-learn module. For more details and tutorials on scikit-learn see:
https://scikit-learn.org/1.3/tutorial/basic/tutorial.html



**Neural Nets and the Kaggle handwritten number data**

We will use a designated training data set from:
https://www.kaggle.com/c/digit-recognizer/data. You can download the data from the above url or from *Canvas*. Each row in the data file consists of a class label (integer between 0 to 9) and a 28*28=784 element vector specifying a gray scale image (size=**nx*ny=28*28**) of a handwritten number (thus total length of each row = 784). Gray scale images (unlike RGB images) just specify the shade of gray with integers between 0 to 255 for each pixel.

1. Load the `mnist_scaled.npz` data file using:
   `np.load()` which returns a Python dictionary.
2. Create a separate vector for the class labels, which are the first entry in each row. Use sth like:
   ```
   y_train = dData['y_train']
   X_train = dData['X_train']
   ```
   Separate the data into training and validation data sets using `train_test_split`.
3. We want to make sure that the data-import worked correctly so a test figure would be helpful. Write a for loop over ~ 5 random indices, get the corresponding vectors describing the image of a number (!without class label!) and plot the matrix using:
   `plt.imshow( [matrix], cmap = plt.cm.Greys`).
   To create the square matrix from the vector use:
   `X_test.reshape( nx,ny)` where nx and ny are the number of pixels in the x and y direction as specified above.

4. Train a MultiLayerPerceptron with the following hyperparameters:
   nn_clf = MLPClassifier()
   Optional key word arguments:

hidden_layer_sizes=100, activation='logistic',solver='sqd',
alpha=0.001 # this is the regularization factor,

learning_rate_init=0.01, max_iter=100, shuffle=True, random_state=1,validation_fraction=0.05

As a side note, the MLP parameters can also be passed to the class directly as a python dictionary, which is much tidier than having to specify each parameter individually:

```
>>>dPar = {'solver': 'sgd', 'learning_rate_init': 1e03, 'shuffle':True,}
>>>nn_clf = MLPClassifer( **dPar)
```

## Tasks:

a) Train the model!
b) Report accuracy, precision and recall based on these parameters!
c) Plot and save the corresponding confusion matrix!
d) Plot and save the misfit (cost)-function as a function of epoch. Scikit-learn again provides a nice short-cut to do this:

plt.plot( o_classifier.loss_curve_, 'ko-', label = ['my_label'])

## Answer the following questions based on the above analysis:

1. Which number is most easily mistaken for a 0,1,2,3,4 … 9?
2. Which number is generally the hardest to predict? (at least for folks with bad handwriting)
3. Which two numbers appear to be most similar?

## Further Analysis:

5. Find hyper-parameters that increase the accuracy to more than 95 percent! Report your hyperparameters!
    1. You can do this by trial and error
    2. You can write a nested for-loop that tests the trade-offs between two hypderparameters at a time. This type of analysis is called a grid-search apparoach which is very commonly used in machine-learning. Hit run and get a cup of coffee while the computer is doing all the work!
6. Write a for loop to explore the trade-offs between training and validation data size. For this purpose you will only use 10% of the original data. Train you Neural Net using a train/validation data split between 5 and 95 percent in 10 percent increments, e.g.:

```
>>for curr_valid_size in np.arange(5, 95+10, 10):
        nn_clf = MLPClassifier( validation_fraction = curr_valid_size)
….
```

Plot the cost functions (loss_curve_) for each data split and describe your observations!